

NLP Assignment 2

- Group No - 28
- Problem Statement - 7
- Dataset Link - <https://www.kaggle.com/datasets/stanfordu/stanford-question-answering-dataset>
- Dataset Name - Stanford Question Answering Dataset

0. Environment Setup

If the packages are already installed, it can be skipped.

```
In [ ]: # Run only if required
        #!uv pip install -q tensorflow pandas numpy matplotlib scikit-learn datasets
```

```
In [1]: import os
import re
import json
import random
import string
from collections import Counter

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.layers import (
    Input, Embedding, GRU, Bidirectional, MultiHeadAttention,
    Concatenate, GlobalMaxPooling1D, Dense, Lambda
)
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam

from sklearn.model_selection import train_test_split

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

Task 1 — Targeted Data Extraction & Embedding Alignment

1.1 Load and parse train-v1.1.json

The code first looks for `train-v1.1.json` locally. If it is not present, it downloads the official SQuAD 1.1 training JSON and then parses the original nested structure:

data → paragraphs → qas → answers

```
In [ ]: import urllib.request

DATA_PATH = "train-v1.1.json"
OFFICIAL_URL = "https://www.kaggle.com/datasets/stanfordu/stanford-question-answer-pairs/train-v1.1.json"

if not os.path.exists(DATA_PATH):
    print("train-v1.1.json not found locally. Downloading SQuAD 1.1...")
    urllib.request.urlretrieve(OFFICIAL_URL, DATA_PATH)

with open(DATA_PATH, "r", encoding="utf-8") as f:
    squad_json = json.load(f)

records = []

for article in squad_json["data"]:
    title = article.get("title", "")
    for paragraph in article["paragraphs"]:
        context = paragraph["context"]
        for qa in paragraph["qas"]:
            question = qa["question"]
            answers = qa.get("answers", [])

            # One row per QA pair; use first reference answer as decoder target.
            # Keep all references for possible inspection.
            if answers:
                records.append({
                    "id": qa.get("id", ""),
                    "title": title,
                    "context": context,
                    "question": question,
                    "answer": answers[0]["text"],
                    "answer_start": answers[0]["answer_start"],
                    "all_answers": [a["text"] for a in answers]
                })

df = pd.DataFrame(records)

print("Total parsed QA pairs:", len(df))
display(df[["context", "question", "answer", "answer_start"]].head())
```

Total parsed QA pairs: 87599

	context	question	answer	answer_start
0	Architecturally, the school has a Catholic cha...	To whom did the Virgin Mary allegedly appear i...	Saint Bernadette Soubirous	515
1	Architecturally, the school has a Catholic cha...	What is in front of the Notre Dame Main Building?	a copper statue of Christ	188
2	Architecturally, the school has a Catholic cha...	The Basilica of the Sacred heart at Notre Dame...	the Main Building	279
3	Architecturally, the school has a Catholic cha...	What is the Grotto at Notre Dame?	a Marian place of prayer and reflection	381
4	Architecturally, the school has a Catholic cha...	What sits on top of the Main Building at Notre...	a golden statue of the Virgin Mary	92

1.2 Filter factoid question categories: Who, Where, When, What

```
In [ ]: QUESTION_TYPES = ("who", "where", "when", "what")

def get_question_type(question):
    first = question.strip().split()[0].lower().rstrip(string.punctuation)
    return first.capitalize() if first in QUESTION_TYPES else None

df["question_type"] = df["question"].apply(get_question_type)
factoid_df = df[df["question_type"].notna()].copy().reset_index(drop=True)

print("Factoid QA pairs:", len(factoid_df))
print("\nQuestion-type distribution:")
print(factoid_df["question_type"].value_counts())
```

Factoid QA pairs: 54523

Question-type distribution:

```
question_type
What      37604
Who       8165
When      5461
Where     3293
Name: count, dtype: int64
```

Practical training subset

A dual-encoder Seq2Seq model is much heavier than ordinary classification.

`MAX_SAMPLES` can be increased if a GPU and enough time/memory are available. The random seed makes the subset reproducible.

```
In [ ]: MAX_SAMPLES = 120000 # Increase if your hardware permits

if len(factoid_df) > MAX_SAMPLES:
    model_df = (
        factoid_df.groupby("question_type", group_keys=False)
```

```

        .apply(lambda x: x.sample(
            n=min(len(x), max(1, MAX_SAMPLES // 4)),
            random_state=SEED
        ))
        .reset_index(drop=True)
    )

    # Fill any remaining slots, if a category was too small
    if len(model_df) < MAX_SAMPLES:
        remaining = factoid_df.drop(model_df.index, errors="ignore")
        extra_n = min(MAX_SAMPLES - len(model_df), len(remaining))
        if extra_n > 0:
            model_df = pd.concat(
                [model_df, remaining.sample(extra_n, random_state=SEED)],
                ignore_index=True
            )
    else:
        model_df = factoid_df.copy()

    model_df = model_df.sample(frac=1, random_state=SEED).reset_index(drop=True)

    print("Rows used for modelling:", len(model_df))
    print(model_df["question_type"].value_counts())

```

Rows used for modelling: 54523

question_type

What 37604

Who 8165

When 5461

Where 3293

Name: count, dtype: int64

1.3 Text normalization

Normalization applies:

- lowercasing,
- punctuation removal,
- whitespace normalization.

For answer generation, `startseq` and `endseq` tokens are later added to each normalized target answer.

```

In [ ]: def normalize_text(text):
        text = str(text).lower()
        text = re.sub(r"^[a-z0-9\s'-]", " ", text)
        text = re.sub(r"\s+", " ", text).strip()
        return text

model_df["context_norm"] = model_df["context"].apply(normalize_text)
model_df["question_norm"] = model_df["question"].apply(normalize_text)
model_df["answer_norm"] = model_df["answer"].apply(normalize_text)

# Remove accidental empty targets
model_df = model_df[model_df["answer_norm"].str.len() > 0].reset_index(drop=True)

display(model_df[[

```

```
"question_type", "context_norm", "question_norm", "answer_norm"
]].head())
```

	question_type	context_norm	question_norm	answer_norm
0	Where	on 27 june the icc issued arrest warrants for ...	where did the july 15 2011 meeting take place	istanbul
1	What	worldwide consumption of paper has risen by 40...	what percentage of wood pulp comes from old-gr...	10
2	What	according to recent estimates 50 of the popula...	what do unrecognized faiths and denominations ...	a registration process
3	What	byu has designated energy conservation product...	what is byu replacing it's incandescent lighti...	fluorescent
4	Where	however since the end of the cold war as the n...	where is the current focus of the canadian mil...	out of area

1.4 Sequence-length distributions and percentile-based thresholds

```
In [ ]: context_lengths = model_df["context_norm"].str.split().str.len()
question_lengths = model_df["question_norm"].str.split().str.len()
answer_lengths = model_df["answer_norm"].str.split().str.len()

print("Context length percentiles:")
print(context_lengths.quantile([0.90, 0.95, 0.99]))
print("\nQuestion length percentiles:")
print(question_lengths.quantile([0.90, 0.95, 0.99]))
print("\nAnswer length percentiles:")
print(answer_lengths.quantile([0.90, 0.95, 0.99]))

plt.figure(figsize=(10, 5))
plt.hist(context_lengths, bins=50, alpha=0.65, label="Contexts")
plt.hist(question_lengths, bins=30, alpha=0.65, label="Questions")
plt.xlabel("Number of tokens")
plt.ylabel("Frequency")
plt.title("Sequence Length Distribution: Contexts vs Questions")
plt.legend()
plt.show()

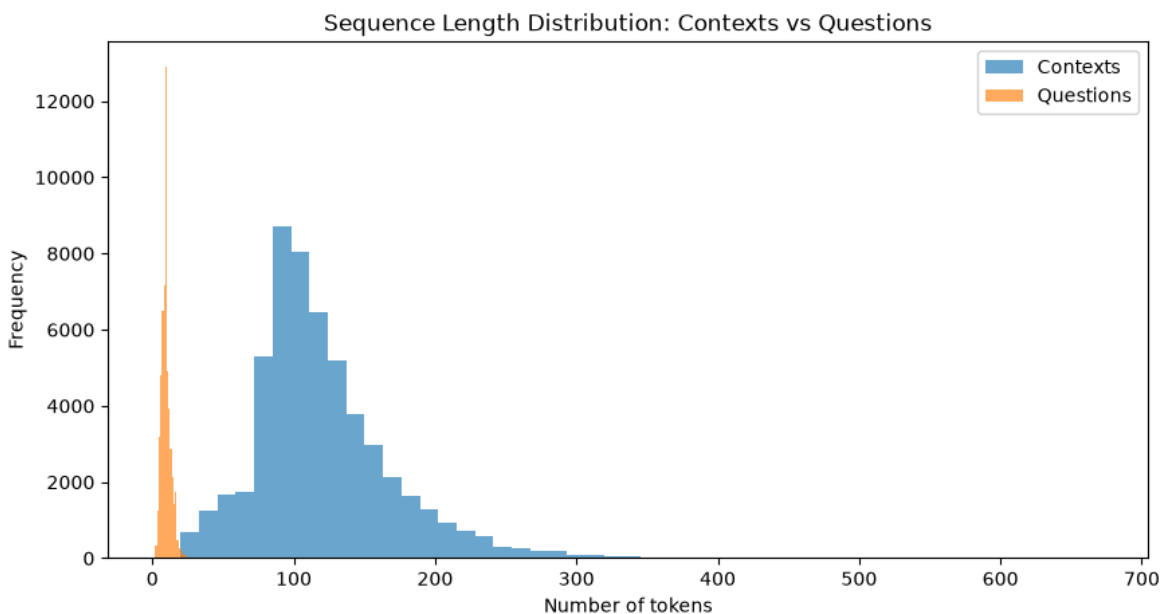
# 95th-percentile coverage, with safe computational caps
MAX_CONTEXT_LEN = min(int(np.ceil(context_lengths.quantile(0.95))), 300)
MAX_QUESTION_LEN = min(int(np.ceil(question_lengths.quantile(0.95))), 40)
MAX_ANSWER_LEN = min(int(np.ceil(answer_lengths.quantile(0.99))) + 2, 30)

print("Chosen MAX_CONTEXT_LEN =", MAX_CONTEXT_LEN)
print("Chosen MAX_QUESTION_LEN =", MAX_QUESTION_LEN)
print("Chosen MAX_ANSWER_LEN =", MAX_ANSWER_LEN)
```

```
Context length percentiles:
0.90    185.0
0.95    215.0
0.99    285.0
Name: context_norm, dtype: float64
```

```
Question length percentiles:
0.90     14.0
0.95     16.0
0.99     20.0
Name: question_norm, dtype: float64
```

```
Answer length percentiles:
0.90      7.0
0.95     10.0
0.99     18.0
Name: answer_norm, dtype: float64
```



```
Chosen MAX_CONTEXT_LEN = 215
Chosen MAX_QUESTION_LEN = 16
Chosen MAX_ANSWER_LEN = 20
```

1.5 Train/validation split, vocabulary, answer-aware training windows and padded sequences

The context and question encoders still use a shared word vocabulary fitted on **training data only**. The decoder no longer predicts one of 50,000 vocabulary words. Instead, it predicts a **position in the context window** and therefore copies the corresponding context token.

For supervised `model.fit`, a 215-token window is positioned so that the gold answer span is visible. This prevents impossible training examples caused by blindly truncating the first 215 tokens.

To avoid using the gold answer location in the final benchmark, EM/F1 evaluation later scans the **full validation context with overlapping 215-token windows** and selects the most confident answer.

```

In [ ]: train_df, val_df = train_test_split(
    model_df,
    test_size=0.20,
    random_state=SEED,
    stratify=model_df["question_type"]
)

VOCAB_LIMIT = 50000

# Decoder input still uses teacher-forced previous answer words.
# The OUTPUT, however, is a context pointer rather than a vocabulary word.
shared_tokenizer = Tokenizer(
    num_words=VOCAB_LIMIT,
    oov_token="<OOV>",
    filters=""
)

training_corpus = (
    train_df["context_norm"].tolist()
    + train_df["question_norm"].tolist()
    + train_df["answer_norm"].tolist()
    + ["startseq endseq"]
)

shared_tokenizer.fit_on_texts(training_corpus)

ctx_tokenizer = shared_tokenizer
q_tokenizer = shared_tokenizer
ans_tokenizer = shared_tokenizer

SHARED_VOCAB = min(
    VOCAB_LIMIT,
    len(shared_tokenizer.word_index) + 1
)

CTX_VOCAB = SHARED_VOCAB
Q_VOCAB = SHARED_VOCAB

# Get the ID exactly as the model sees it after applying num_words=50000
start_id = shared_tokenizer.texts_to_sequences(
    ["startseq"]
)[0][0]

end_word_id = shared_tokenizer.texts_to_sequences(
    ["endseq"]
)[0][0]

print("start_id used by model:", start_id)
print("end_word_id used by model:", end_word_id)
print("SHARED_VOCAB:", SHARED_VOCAB)

start_id used by model: 1
end_word_id used by model: 1
SHARED_VOCAB: 50000

```

```

In [ ]: # =====
# 2. ANSWER-AWARE SUPERVISED CONTEXT WINDOWS
# =====

MAX_CONTENT_ANSWER_TOKENS = MAX_ANSWER_LEN - 2

```

```

def find_token_subsequence(context_tokens, answer_tokens):
    """Return the first exact normalized-token occurrence."""
    if not answer_tokens or len(answer_tokens) > len(context_tokens):
        return None

    last_start = len(context_tokens) - len(answer_tokens)

    for start in range(last_start + 1):
        if context_tokens[start:start + len(answer_tokens)] == answer_tokens:
            return start

    return None

def make_answer_aware_window(
    context_tokens,
    answer_start_token,
    answer_length,
    max_context_len
):
    """
    Create a fixed-length token window containing the complete answer.
    The answer is approximately centered when enough left/right context exists.
    """
    if len(context_tokens) <= max_context_len:
        window_start = 0
    else:
        free_context = max_context_len - answer_length
        left_context = max(0, free_context // 2)

        window_start = answer_start_token - left_context
        window_start = max(0, window_start)
        window_start = min(
            window_start,
            len(context_tokens) - max_context_len
        )

        # Defensive correction: guarantee the answer end is inside the window.
        answer_end = answer_start_token + answer_length
        if answer_end > window_start + max_context_len:
            window_start = answer_end - max_context_len

    window_end = min(
        window_start + max_context_len,
        len(context_tokens)
    )

    window_tokens = context_tokens[window_start:window_end]
    local_answer_start = answer_start_token - window_start

    pointer_positions = list(
        range(
            local_answer_start,
            local_answer_start + answer_length
        )
    )

    return window_tokens, window_start, pointer_positions

```

```

def build_supervised_pointer_frame(frame):
    """
    Build supervised fixed windows and pointer labels.

    Rows are removed only when:
    - normalized answer is longer than the chosen decoder capacity, or
    - the normalized answer cannot be located in normalized context.
    """
    records = []
    dropped_long = 0
    dropped_not_found = 0

    for _, row in frame.iterrows():
        context_tokens = row["context_norm"].split()
        answer_tokens = row["answer_norm"].split()

        if len(answer_tokens) > MAX_CONTENT_ANSWER_TOKENS:
            dropped_long += 1
            continue

        answer_start_token = find_token_subsequence(
            context_tokens,
            answer_tokens
        )

        if answer_start_token is None:
            dropped_not_found += 1
            continue

        (
            window_tokens,
            window_start,
            pointer_positions
        ) = make_answer_aware_window(
            context_tokens,
            answer_start_token,
            len(answer_tokens),
            MAX_CONTEXT_LEN
        )

        record = row.to_dict()
        record["context_window_norm"] = " ".join(window_tokens)
        record["context_window_tokens"] = window_tokens
        record["window_start_token"] = window_start
        record["answer_pointer_positions"] = pointer_positions

        records.append(record)

    out = pd.DataFrame(records).reset_index(drop=True)

    print(
        f"Kept {len(out):,}/{len(frame):,} rows | "
        f"dropped long answers={dropped_long:,}, "
        f"answer-not-found={dropped_not_found:,}"
    )

    return out

```

```

print("Training windows:")
train_df = build_supervised_pointer_frame(train_df)

print("\nValidation windows used for val_loss:")
val_df = build_supervised_pointer_frame(val_df)

print(
    "\nNOTE: final EM/F1 will use sliding windows over each FULL validation "
    "context, not these gold-positioned validation windows."
)

```

Training windows:

Kept 42,773/43,609 rows | dropped long answers=425, answer-not-found=411

Validation windows used for val_loss:

Kept 10,663/10,903 rows | dropped long answers=108, answer-not-found=132

NOTE: final EM/F1 will use sliding windows over each FULL validation context, not these gold-positioned validation windows.

```

In [ ]: # =====
# 3. CONVERT INPUT TEXT TO SEQUENCES AND PAD
# =====

# Supervised context windows
Xctx_train_seq = ctx_tokenizer.texts_to_sequences(
    train_df["context_window_norm"]
)

Xctx_val_seq = ctx_tokenizer.texts_to_sequences(
    val_df["context_window_norm"]
)

# Questions
Xq_train_seq = q_tokenizer.texts_to_sequences(
    train_df["question_norm"]
)

Xq_val_seq = q_tokenizer.texts_to_sequences(
    val_df["question_norm"]
)

# Teacher-forcing decoder input:
# startseq + gold answer words
decoder_train_text = train_df["answer_norm"].apply(
    lambda x: f"startseq {x}"
)

decoder_val_text = val_df["answer_norm"].apply(
    lambda x: f"startseq {x}"
)

Xans_train_seq = ans_tokenizer.texts_to_sequences(
    decoder_train_text
)

Xans_val_seq = ans_tokenizer.texts_to_sequences(
    decoder_val_text
)

```

```

Xctx_train = pad_sequences(
    Xctx_train_seq,
    maxlen=MAX_CONTEXT_LEN,
    padding="post",
    truncating="post"
)

Xctx_val = pad_sequences(
    Xctx_val_seq,
    maxlen=MAX_CONTEXT_LEN,
    padding="post",
    truncating="post"
)

Xq_train = pad_sequences(
    Xq_train_seq,
    maxlen=MAX_QUESTION_LEN,
    padding="post",
    truncating="post"
)

Xq_val = pad_sequences(
    Xq_val_seq,
    maxlen=MAX_QUESTION_LEN,
    padding="post",
    truncating="post"
)

Xans_train = pad_sequences(
    Xans_train_seq,
    maxlen=MAX_ANSWER_LEN - 1,
    padding="post",
    truncating="post"
)

Xans_val = pad_sequences(
    Xans_val_seq,
    maxlen=MAX_ANSWER_LEN - 1,
    padding="post",
    truncating="post"
)

print("Context train      :", Xctx_train.shape)
print("Question train     :", Xq_train.shape)
print("Decoder input train:", Xans_train.shape)

```

```

Context train      : (42773, 215)
Question train     : (42773, 16)
Decoder input train: (42773, 19)

```

```

In [ ]: # =====
# 4. POINTER TARGETS
# =====

# Pointer output classes:
# 0 ... MAX_CONTEXT_LEN-1 -> copy this context position
# MAX_CONTEXT_LEN         -> END
END_POINTER_ID = MAX_CONTEXT_LEN
NUM_POINTER_CLASSES = MAX_CONTEXT_LEN + 1

```

```

ENDSEQ_LOSS_WEIGHT = 0.10

def build_pointer_targets(frame):
    steps = MAX_ANSWER_LEN - 1

    # Fill unused target slots with END. Their loss weight remains zero.
    y_pointer = np.full(
        (len(frame), steps),
        END_POINTER_ID,
        dtype=np.int32
    )

    loss_weights = np.zeros(
        (len(frame), steps),
        dtype=np.float32
    )

    for i, positions in enumerate(
        frame["answer_pointer_positions"]
    ):
        positions = list(positions)
        answer_len = len(positions)

        if answer_len > steps - 1:
            raise ValueError(
                "Answer exceeds decoder capacity. "
                "This row should have been filtered earlier."
            )

        # Copy answer tokens from their exact local context positions.
        y_pointer[i, :answer_len] = positions
        loss_weights[i, :answer_len] = 1.0

        # Predict END immediately after the final answer token.
        y_pointer[i, answer_len] = END_POINTER_ID
        loss_weights[i, answer_len] = ENDSEQ_LOSS_WEIGHT

    return y_pointer, loss_weights

y_train, train_weights = build_pointer_targets(train_df)
y_val, val_weights = build_pointer_targets(val_df)

print("Pointer target train:", y_train.shape)
print("Pointer classes      :", NUM_POINTER_CLASSES)
print("END pointer id       :", END_POINTER_ID)

content_positions = int(np.sum(train_weights == 1.0))
end_positions = int(np.sum(train_weights == ENDSEQ_LOSS_WEIGHT))
ignored_positions = int(np.sum(train_weights == 0.0))

print(
    f"Weighted positions -> content={content_positions:}, "
    f"END={end_positions:}, ignored={ignored_positions:}"
)

```

Pointer target train: (42773, 19)
Pointer classes : 216
END pointer id : 215
Weighted positions -> content=134,654, END=42,773, ignored=635,260

```
In [ ]: # =====
# 5. POINTER / COPY DIAGNOSTIC
# =====

print(
    "Old vocabulary classifier classes :",
    50000
)

print(
    "New pointer output classes      :",
    NUM_POINTER_CLASSES
)

print(
    "Output-class reduction          :",
    f"{50000 / NUM_POINTER_CLASSES:.1f}x"
)

print(
    "\nThe output decoder can copy ANY raw token present in the selected "
    "context window, even when that word maps to <OOV> in the input tokenizer."
)
```

Old vocabulary classifier classes : 50000
New pointer output classes : 216
Output-class reduction : 231.5x

The output decoder can copy ANY raw token present in the selected context window, even when that word maps to <OOV> in the input tokenizer.

```
In [ ]: # =====
# 6. INPUT-VOCABULARY OOV DIAGNOSTIC
# =====
#
# OOV tokens may still exist in the encoder input, but unlike the old
# vocabulary-softmax model they are no longer impossible decoder outputs:
# the pointer predicts a context POSITION and evaluation copies the raw token.

def calculate_input_vocab_oov_rate(texts, tokenizer, vocab_limit):
    total_tokens = 0
    oov_tokens = 0

    for text in texts:
        for word in text.split():
            total_tokens += 1
            idx = tokenizer.word_index.get(word)

            if idx is None or idx >= vocab_limit:
                oov_tokens += 1

    rate = oov_tokens / total_tokens if total_tokens else 0.0
    return oov_tokens, total_tokens, rate
```

```

val_oov, val_total, val_oov_rate = calculate_input_vocab_oov_rate(
    val_df["answer_norm"],
    shared_tokenizer,
    VOCAB_LIMIT
)

print(
    f"Validation answer tokens outside input 50k vocabulary: "
    f"{val_oov}/{val_total} ({val_oov_rate:.2%})"
)

print(
    "These tokens remain copyable because pointer outputs are context positions."
)

```

Validation answer tokens outside input 50k vocabulary: 1700/33524 (5.07%)
 These tokens remain copyable because pointer outputs are context positions.

```

In [ ]: # =====
# 7. VERIFY GOLD POINTERS RECOVER THE ANSWER
# =====

for i in range(min(5, len(train_df))):
    tokens = train_df.iloc[i]["context_window_tokens"]
    ptrs = train_df.iloc[i]["answer_pointer_positions"]

    recovered = " ".join(
        tokens[p]
        for p in ptrs
    )

    print(
        f"{i}: target='{train_df.iloc[i]['answer_norm']}' "
        f"| recovered='{recovered}'"
    )

```

0: target='alfred tarski' | recovered='alfred tarski'
 1: target='its cuisine and efficiency of transit' | recovered='its cuisine and ef
 ficiency of transit'
 2: target='brahman soul and matter' | recovered='brahman soul and matter'
 3: target='since the early 1970s' | recovered='since the early 1970s'
 4: target='november 1871' | recovered='november 1871'

```

In [ ]: # =====
# 8. RARE ENTITY CHECK
# =====

words_to_check = [
    "alfred",
    "tarski",
    "private",
    "henry",
    "wyatt",
    "yayousu",
    "kiss",
    "leo",
    "tolstoy"
]

for word in words_to_check:
    idx = shared_tokenizer.word_index.get(word)

```

```

if idx is None:
    input_status = "UNSEEN"
elif idx >= VOCAB_LIMIT:
    input_status = f"INPUT-OOV (rank={idx})"
else:
    input_status = f"input id={idx}"

print(
    f"{word:12} -> {input_status:24} | "
    "pointer output: COPYABLE when present in context"
)

```

```

alfred      -> input id=3618          | pointer output: COPYABLE when present
in context
tarski      -> input id=40340        | pointer output: COPYABLE when present
in context
private     -> input id=465          | pointer output: COPYABLE when present
in context
henry       -> input id=806          | pointer output: COPYABLE when present
in context
wyatt       -> input id=11917        | pointer output: COPYABLE when present
in context
yayousu     -> INPUT-OOV (rank=74847) | pointer output: COPYABLE when present
in context
kiss        -> input id=13266        | pointer output: COPYABLE when present
in context
leo         -> input id=8791         | pointer output: COPYABLE when present
in context
tolstoy     -> INPUT-OOV (rank=76607) | pointer output: COPYABLE when present
in context

```

```

In [ ]: # Task-1 observation / inference
context_coverage = (context_lengths <= MAX_CONTEXT_LEN).mean() * 100
question_coverage = (question_lengths <= MAX_QUESTION_LEN).mean() * 100

print("OBSERVATION & INFERENCE – TASK 1")
print(f"- {len(factoid_df):,} factoid Who/Where/When/What questions were obtained")
print(f"- The 215-token threshold directly covers about {context_coverage:.2f}%")
print(f"- The {MAX_QUESTION_LEN}-token threshold covers about {question_coverage:.2f}%")
print("- During supervised training, the fixed context window is positioned to cover the full question,")
print(" preventing answer truncation while preserving the selected fixed-length context window")
print("- The final EM/F1 benchmark later uses overlapping windows over the full question")
print(" so the gold answer location is not used to choose the evaluation window")
print("- Encoder inputs use a shared training-only vocabulary; decoder outputs a shared vocabulary")
print(" which avoids the closed-vocabulary answer-generation bottleneck.")

```

OBSERVATION & INFERENCE – TASK 1

- 54,523 factoid Who/Where/When/What questions were obtained.
- The 215-token threshold directly covers about 95.10% of contexts.
- The 16-token threshold covers about 96.24% of questions.
- During supervised training, the fixed context window is positioned to contain the gold span, preventing answer truncation while preserving the selected fixed-length threshold.
- The final EM/F1 benchmark later uses overlapping windows over the full validation context, so the gold answer location is not used to choose the evaluation window.
- Encoder inputs use a shared training-only vocabulary; decoder outputs are context positions, which avoids the closed-vocabulary answer-generation bottleneck.

Task 2 — Dual-Encoder Architecture with Cross-Attention

2.1 Model design

- **Context encoder:** shared embedding → Bidirectional GRU
- **Question encoder:** shared embedding → Bidirectional GRU
- **Cross-attention:** every context token attends to the encoded question (215×16)
- **Fusion:** original context representation + question-aware context representation
- **Decoder:** teacher-forced answer embedding → GRU initialized from the fused context
- **Dynamic decoder attention:** decoder states attend over the question-aware context
- **Pointer output:** each decoder step predicts one of the 215 context positions or **END**

The pointer output preserves the requested iterative Seq2Seq decoder while replacing the expensive 50,000-way word softmax with a compact context-copy distribution.

```
In [ ]: # Training objective:
#
# - Sparse Categorical Cross-Entropy is applied to pointer classes.
# - Answer-token pointer steps receive weight 1.0.
# - The END pointer receives weight 0.10.
# - Unused padded decoder positions receive weight 0.0.
#
# Accuracy reported below excludes END and therefore measures whether
# actual answer-content positions are selected correctly.
```

```
In [ ]: # =====
# HYPERPARAMETERS
# =====

EMBED_DIM = 128
ENC_UNITS = 128
DEC_UNITS = 256
NUM_HEADS = 4
```

```

KEY_DIM = 64

# =====
# INPUTS
# =====

ctx_input = Input(
    shape=(MAX_CONTEXT_LEN,),
    name="context_input"
)

q_input = Input(
    shape=(MAX_QUESTION_LEN,),
    name="question_input"
)

ans_input = Input(
    shape=(MAX_ANSWER_LEN - 1,),
    name="answer_input"
)

# =====
# SHARED INPUT EMBEDDING
# =====

shared_embedding = Embedding(
    input_dim=SHARED_VOCAB,
    output_dim=EMBED_DIM,
    mask_zero=True,
    name="shared_embedding"
)

ctx_emb = shared_embedding(ctx_input)
q_emb = shared_embedding(q_input)
ans_emb = shared_embedding(ans_input)

# =====
# DUAL BiGRU ENCODERS
# =====

ctx_enc = Bidirectional(
    GRU(
        ENC_UNITS,
        return_sequences=True,
        dropout=0.2
    ),
    name="context_bigru"
)(ctx_emb)

q_enc = Bidirectional(
    GRU(
        ENC_UNITS,
        return_sequences=True,
        dropout=0.2
    ),
    name="question_bigru"
)(q_emb)

```

```

# =====
# SCALED DOT-PRODUCT CROSS-ATTENTION
# Context positions query question positions.
# =====

q_padding_mask = Lambda(
    lambda x: tf.not_equal(x, 0),
    output_shape=(MAX_QUESTION_LEN,),
    name="question_padding_mask"
)(q_input)

cross_attention_logits = Lambda(
    lambda xs: (
        tf.matmul(
            xs[0],
            xs[1],
            transpose_b=True
        )
        / tf.sqrt(
            tf.cast(
                tf.shape(xs[0])[-1],
                xs[0].dtype
            )
        )
    ),
    output_shape=(MAX_CONTEXT_LEN, MAX_QUESTION_LEN),
    name="cross_attention_logits"
)([
    ctx_enc,
    q_enc
])

masked_cross_attention_logits = Lambda(
    lambda xs: tf.where(
        tf.expand_dims(xs[1], axis=1),
        xs[0],
        tf.cast(-1e9, xs[0].dtype)
    ),
    output_shape=(MAX_CONTEXT_LEN, MAX_QUESTION_LEN),
    name="masked_cross_attention_logits"
)([
    cross_attention_logits,
    q_padding_mask
])

cross_attention_weights = Lambda(
    lambda x: tf.nn.softmax(x, axis=-1),
    output_shape=(MAX_CONTEXT_LEN, MAX_QUESTION_LEN),
    name="cross_attention_weights"
)(masked_cross_attention_logits)

cross_attended = Lambda(
    lambda xs: tf.matmul(xs[0], xs[1]),
    output_shape=(MAX_CONTEXT_LEN, ENC_UNITS * 2),
    name="cross_attended_context"
)([
    cross_attention_weights,
    q_enc
])

```

```

])

attention_scores = Lambda(
    lambda x: tf.expand_dims(x, axis=1),
    output_shape=(1, MAX_CONTEXT_LEN, MAX_QUESTION_LEN),
    name="cross_attention_scores"
)(cross_attention_weights)

question_aware_context = Concatenate(
    axis=-1,
    name="question_aware_context"
)([
    ctx_enc,
    cross_attended
])

# =====
# MASKED CONTEXT POOLING -> DECODER INITIAL STATE
# =====

def masked_max_pool(inputs):
    x, token_ids = inputs
    mask = tf.not_equal(token_ids, 0)
    mask = tf.expand_dims(mask, axis=-1)

    very_negative = tf.cast(
        -1e9,
        x.dtype
    )

    masked_x = tf.where(
        mask,
        x,
        very_negative
    )

    return tf.reduce_max(
        masked_x,
        axis=1
    )

context_summary = Lambda(
    masked_max_pool,
    output_shape=(ENC_UNITS * 4,),
    mask=lambda inputs, mask: None,
    name="context_summary_pooling"
)([
    question_aware_context,
    ctx_input
])

decoder_init = Dense(
    DEC_UNITS,
    activation="tanh",
    name="decoder_initial_state"
)(context_summary)

```

```

# =====
# AUTOREGRESSIVE DECODER GRU
# =====

decoder_seq = GRU(
    DEC_UNITS,
    return_sequences=True,
    name="answer_decoder_gru"
)(
    ans_emb,
    initial_state=decoder_init
)

# =====
# DYNAMIC DECODER -> CONTEXT ATTENTION
# =====

ctx_padding_mask = Lambda(
    lambda x: tf.not_equal(x, 0),
    output_shape=(MAX_CONTEXT_LEN,),
    name="context_padding_mask"
)(ctx_input)

decoder_attention_mask = Lambda(
    lambda x: tf.repeat(
        tf.expand_dims(x, axis=1),
        repeats=MAX_ANSWER_LEN - 1,
        axis=1
    ),
    output_shape=(MAX_ANSWER_LEN - 1, MAX_CONTEXT_LEN),
    name="decoder_attention_mask"
)(ctx_padding_mask)

decoder_attention = MultiHeadAttention(
    num_heads=NUM_HEADS,
    key_dim=KEY_DIM,
    dropout=0.1,
    name="decoder_context_attention"
)

decoder_context = decoder_attention(
    query=decoder_seq,
    value=question_aware_context,
    key=question_aware_context,
    attention_mask=decoder_attention_mask
)

decoder_features = Concatenate(
    axis=-1,
    name="decoder_context_features"
)([
    decoder_seq,
    decoder_context
])

decoder_features = Dense(
    DEC_UNITS,
    activation="tanh",
    name="decoder_fusion_projection"
)

```

```

)(decoder_features)

# =====
# POINTER OUTPUT
#
# Instead of Dense(50000), score each decoder step against each
# of the 215 context positions. Add one Learned END score.
# =====

pointer_context = Dense(
    DEC_UNITS,
    use_bias=False,
    name="pointer_context_projection"
)(question_aware_context)

pointer_decoder = Dense(
    DEC_UNITS,
    use_bias=False,
    name="pointer_decoder_projection"
)(decoder_features)

pointer_context_logits = Lambda(
    lambda xs: (
        tf.matmul(
            xs[0],
            xs[1],
            transpose_b=True
        )
        / tf.sqrt(
            tf.cast(
                tf.shape(xs[0])[-1],
                xs[0].dtype
            )
        )
    ),
    output_shape=(MAX_ANSWER_LEN - 1, MAX_CONTEXT_LEN),
    name="pointer_context_logits"
)([
    pointer_decoder,
    pointer_context
])

pointer_mask = Lambda(
    lambda x: tf.repeat(
        tf.expand_dims(
            tf.not_equal(x, 0),
            axis=1
        ),
        repeats=MAX_ANSWER_LEN - 1,
        axis=1
    ),
    output_shape=(MAX_ANSWER_LEN - 1, MAX_CONTEXT_LEN),
    name="pointer_context_mask"
)(ctx_input)

masked_pointer_context_logits = Lambda(
    lambda xs: tf.where(
        xs[1],
        xs[0],

```

```

        tf.cast(-1e9, xs[0].dtype)
    ),
    output_shape=(MAX_ANSWER_LEN - 1, MAX_CONTEXT_LEN),
    name="masked_pointer_context_logits"
)([
    pointer_context_logits,
    pointer_mask
])

end_pointer_logit = Dense(
    1,
    name="end_pointer_logit"
)(decoder_features)

pointer_logits = Concatenate(
    axis=-1,
    name="pointer_logits"
)([
    masked_pointer_context_logits,
    end_pointer_logit
])

pointer_probs = Lambda(
    lambda x: tf.nn.softmax(
        x,
        axis=-1
    ),
    output_shape=(
        MAX_ANSWER_LEN - 1,
        NUM_POINTER_CLASSES
    ),
    name="pointer_probabilities"
)(pointer_logits)

# =====
# MAIN MODEL
# =====

qa_model = Model(
    inputs=[
        ctx_input,
        q_input,
        ans_input
    ],
    outputs=pointer_probs,
    name="DualEncoderCrossAttentionPointerQA"
)

# =====
# CROSS-ATTENTION EXTRACTION MODEL FOR TASK 3
# =====

attention_model = Model(
    inputs=[
        ctx_input,
        q_input
    ],
    outputs=attention_scores,

```

```

        name="CrossAttentionExtractor"
    )

# =====
# CONTENT POINTER ACCURACY
# Excludes END targets. Padded targets are also filled with END.
# =====

@tf.keras.utils.register_keras_serializable(package="QA")
class PointerContentAccuracy(tf.keras.metrics.Metric):

    def __init__(
        self,
        end_pointer_id,
        name="accuracy",
        **kwargs
    ):
        super().__init__(
            name=name,
            **kwargs
        )

        self.end_pointer_id = int(
            end_pointer_id
        )

        self.correct_positions = self.add_weight(
            name="correct_positions",
            initializer="zeros"
        )

        self.total_positions = self.add_weight(
            name="total_positions",
            initializer="zeros"
        )

    def update_state(
        self,
        y_true,
        y_pred,
        sample_weight=None
    ):
        y_true = tf.cast(
            y_true,
            tf.int32
        )

        pred = tf.argmax(
            y_pred,
            axis=-1,
            output_type=tf.int32
        )

        content_mask = tf.not_equal(
            y_true,
            self.end_pointer_id
        )

        correct = tf.logical_and(

```

```

        content_mask,
        tf.equal(
            y_true,
            pred
        )
    )

    self.correct_positions.assign_add(
        tf.reduce_sum(
            tf.cast(
                correct,
                self.dtype
            )
        )
    )

    self.total_positions.assign_add(
        tf.reduce_sum(
            tf.cast(
                content_mask,
                self.dtype
            )
        )
    )

def result(self):
    return tf.math.divide_no_nan(
        self.correct_positions,
        self.total_positions
    )

def reset_state(self):
    self.correct_positions.assign(0.0)
    self.total_positions.assign(0.0)

def get_config(self):
    config = super().get_config()

    config.update({
        "end_pointer_id": self.end_pointer_id
    })

    return config

qa_model.compile(
    optimizer=Adam(
        learning_rate=3e-4,
        clipnorm=1.0
    ),

    loss=tf.keras.losses.SparseCategoricalCrossentropy(),

    metrics=[
        PointerContentAccuracy(
            end_pointer_id=END_POINTER_ID,
            name="accuracy"
        )
    ]
)

```

```
qa_model.summary()
```

```
E0000 00:00:1787457063.291025    3396 cuda_platform.cc:52] failed call to cuInit:
INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
/home/cloud/nlp_gpu/lib64/python3.12/site-packages/keras/src/layers/layer.py:103
9: UserWarning: Layer 'cross_attention_logits' (of type Lambda) was passed an inp
ut with a mask attached to it. However, this layer does not support masking and w
ill therefore destroy the mask information. Downstream layers will not see the ma
sk.
    warnings.warn(
/home/cloud/nlp_gpu/lib64/python3.12/site-packages/keras/src/layers/layer.py:103
9: UserWarning: Layer 'cross_attended_context' (of type Lambda) was passed an inp
ut with a mask attached to it. However, this layer does not support masking and w
ill therefore destroy the mask information. Downstream layers will not see the ma
sk.
    warnings.warn(
/home/cloud/nlp_gpu/lib64/python3.12/site-packages/keras/src/layers/layer.py:103
9: UserWarning: Layer 'pointer_context_logits' (of type Lambda) was passed an inp
ut with a mask attached to it. However, this layer does not support masking and w
ill therefore destroy the mask information. Downstream layers will not see the ma
sk.
    warnings.warn(
/home/cloud/nlp_gpu/lib64/python3.12/site-packages/keras/src/layers/layer.py:103
9: UserWarning: Layer 'pointer_probabilities' (of type Lambda) was passed an inp
ut with a mask attached to it. However, this layer does not support masking and w
ill therefore destroy the mask information. Downstream layers will not see the mas
k.
    warnings.warn(
```

Model: "DualEncoderCrossAttentionPointerQA"

Layer (type)	Output Shape	Param #	Connected to
answer_input (InputLayer)	(None, 19)	0	-
context_input (InputLayer)	(None, 215)	0	-
question_input (InputLayer)	(None, 16)	0	-
shared_embedding (Embedding)	(None, 19, 128)	6,400,000	context_input[0]... question_input[0]... answer_input[0][...
not_equal (NotEqual)	(None, 215)	0	context_input[0]...
not_equal_1 (NotEqual)	(None, 16)	0	question_input[0]...
context_bigru (Bidirectional)	(None, 215, 256)	198,144	shared_embedding... not_equal[0][0]
question_bigru (Bidirectional)	(None, 16, 256)	198,144	shared_embedding... not_equal_1[0][0]
cross_attention_lo... (Lambda)	(None, 215, 16)	0	context_bigru[0]... question_bigru[0]...
question_padding_m... (Lambda)	(None, 16)	0	question_input[0]...
masked_cross_atten... (Lambda)	(None, 215, 16)	0	cross_attention_... question_padding...
cross_attention_we... (Lambda)	(None, 215, 16)	0	masked_cross_att...
cross_attended_con... (Lambda)	(None, 215, 256)	0	cross_attention_... question_bigru[0]...
question_aware_con... (Concatenate)	(None, 215, 512)	0	context_bigru[0]... cross_attended_c...
context_summary_po... (Lambda)	(None, 512)	0	question_aware_c... context_input[0]...
decoder_initial_st... (Dense)	(None, 256)	131,328	context_summary_...
context_padding_ma... (Lambda)	(None, 215)	0	context_input[0]...
answer_decoder_gru (GRU)	(None, 19, 256)	296,448	shared_embedding... decoder_initial_...
decoder_attention_... (Lambda)	(None, 19, 215)	0	context_padding_...

decoder_context_at... (MultiHeadAttentio...	(None, 19, 256)	394,240	decoder_attentio... question_aware_c... answer_decoder_g... question_aware_c...
not_equal_2 (NotEqual)	(None, 19)	0	answer_input[0][...
decoder_context_fe... (Concatenate)	(None, 19, 512)	0	answer_decoder_g... decoder_context_...
expand_dims_1 (ExpandDims)	(None, 19, 1)	0	not_equal_2[0][0]
zeros_like_1 (ZerosLike)	(None, 19, 256)	0	answer_decoder_g...
expand_dims_2 (ExpandDims)	(None, 19, 1)	0	not_equal_2[0][0]
zeros_like_2 (ZerosLike)	(None, 19, 256)	0	decoder_context_...
decoder_fusion_pro... (Dense)	(None, 19, 256)	131,328	decoder_context_...
logical_or_1 (LogicalOr)	(None, 19, 256)	0	expand_dims_1[0]... zeros_like_1[0][...
logical_or_2 (LogicalOr)	(None, 19, 256)	0	expand_dims_2[0]... zeros_like_2[0][...
pointer_decoder_pr... (Dense)	(None, 19, 256)	65,536	decoder_fusion_p...
pointer_context_pr... (Dense)	(None, 215, 256)	131,072	question_aware_c...
concatenate_1 (Concatenate)	(None, 19, 512)	0	logical_or_1[0][...] logical_or_2[0][...
pointer_context_lo... (Lambda)	(None, 19, 215)	0	pointer_decoder_... pointer_context_...
pointer_context_ma... (Lambda)	(None, 19, 215)	0	context_input[0]...
end_pointer_logit (Dense)	(None, 19, 1)	257	decoder_fusion_p...
any_1 (Any)	(None, 19)	0	concatenate_1[0]...
masked_pointer_con... (Lambda)	(None, 19, 215)	0	pointer_context_... pointer_context_...
expand_dims_3 (ExpandDims)	(None, 19, 1)	0	any_1[0][0]
zeros_like_3	(None, 19, 1)	0	end_pointer_logi...

(ZerosLike)			
ones_like_1 (OnesLike)	(None, 19, 215)	0	masked_pointer_c...
logical_or_3 (LogicalOr)	(None, 19, 1)	0	expand_dims_3[0]... zeros_like_3[0][...
concatenate_2 (Concatenate)	(None, 19, 216)	0	ones_like_1[0][0]... logical_or_3[0][...
pointer_logits (Concatenate)	(None, 19, 216)	0	masked_pointer_c... end_pointer_logi...
any_2 (Any)	(None, 19)	0	concatenate_2[0]...
pointer_probabilit... (Lambda)	(None, 19, 216)	0	pointer_logits[0]... any_2[0][0]

Total params: 7,946,497 (30.31 MB)

Trainable params: 7,946,497 (30.31 MB)

Non-trainable params: 0 (0.00 B)

```
In [ ]: print("Context encoder shape      :", ctx_enc.shape)
print("Question encoder shape        :", q_enc.shape)
print("Cross-attended shape          :", cross_attended.shape)
print("Question-aware context shape:", question_aware_context.shape)
print("Attention scores shape         :", attention_scores.shape)
print("Decoder sequence shape         :", decoder_seq.shape)
print("Pointer output shape           :", pointer_probs.shape)

print(
    "\nExpected pointer classes:",
    NUM_POINTER_CLASSES,
    "= 215 context positions + END"
)
```

```
Context encoder shape      : (None, 215, 256)
Question encoder shape     : (None, 16, 256)
Cross-attended shape       : (None, 215, 256)
Question-aware context shape: (None, 215, 512)
Attention scores shape      : (None, 1, 215, 16)
Decoder sequence shape     : (None, 19, 256)
Pointer output shape       : (None, 19, 216)
```

Expected pointer classes: 216 = 215 context positions + END

2.2 Train using Adam + Sparse Categorical Cross-Entropy

The target at each decoder step is now a **context-position class** rather than a vocabulary word. Sparse Categorical Cross-Entropy is therefore retained exactly as required, while temporal sample weights ignore padding and reduce the universal **END** contribution.

```
In [ ]: # =====
# CALLBACKS
# =====

callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        filepath="best_pointer_qa_v7.keras",
        monitor="val_loss",
        save_best_only=True,
        mode="min",
        verbose=1
    ),

    tf.keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=2,
        min_lr=1e-5,
        mode="min",
        verbose=1
    )
]
```

```
In [ ]: print("Xctx_train :", Xctx_train.shape)
print("Xq_train   :", Xq_train.shape)
print("Xans_train :", Xans_train.shape)
print("y_train    :", y_train.shape)

print("\nInput vocabulary      :", SHARED_VOCAB)
print("Pointer output classes :", NUM_POINTER_CLASSES)
print("END pointer id         :", END_POINTER_ID)

print(
    "\nThe output classifier has been reduced from 50,000 word classes "
    f"to {NUM_POINTER_CLASSES} pointer classes."
)

print(
    "Training/validation accuracy measures exact ANSWER CONTEXT-POSITION "
    "selection and excludes END."
)
```

```
Xctx_train : (42773, 215)
Xq_train   : (42773, 16)
Xans_train : (42773, 19)
y_train    : (42773, 19)
```

```
Input vocabulary      : 50000
Pointer output classes : 216
END pointer id        : 215
```

The output classifier has been reduced from 50,000 word classes to 216 pointer classes.

Training/validation accuracy measures exact ANSWER CONTEXT-POSITION selection and excludes END.

```
In [ ]: # =====
# TRAINING
# =====
```

```

EPOCHS = 15
BATCH_SIZE = 16

history = qa_model.fit(
    [
        Xctx_train,
        Xq_train,
        Xans_train
    ],
    y_train,

    sample_weight=train_weights,

    validation_data=(
        [
            Xctx_val,
            Xq_val,
            Xans_val
        ],
        y_val,
        val_weights
    ),

    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=callbacks,
    verbose=1
)

```

Epoch 1/15

E0000 00:00:1787457069.484735 3396 util.cc:131] oneDNN supports DT_BOOL only on platforms with AVX-512. Falling back to the default Eigen-based implementation if present.

2674/2674 ————— 0s 89ms/step - accuracy: 0.1623 - loss: 0.5767
Epoch 1: val_loss improved from None to 0.40001, saving model to best_pointer_qa_v7.keras

Epoch 1: finished saving model to best_pointer_qa_v7.keras
2674/2674 ————— 260s 95ms/step - accuracy: 0.1623 - loss: 0.5767 - val_accuracy: 0.3970 - val_loss: 0.4000 - learning_rate: 3.0000e-04
Epoch 2/15

2674/2674 ————— 0s 89ms/step - accuracy: 0.5212 - loss: 0.3123
Epoch 2: val_loss improved from 0.40001 to 0.33005, saving model to best_pointer_qa_v7.keras

Epoch 2: finished saving model to best_pointer_qa_v7.keras
2674/2674 ————— 253s 95ms/step - accuracy: 0.5212 - loss: 0.3123 - val_accuracy: 0.5117 - val_loss: 0.3300 - learning_rate: 3.0000e-04
Epoch 3/15

2674/2674 ————— 0s 89ms/step - accuracy: 0.6453 - loss: 0.2332
Epoch 3: val_loss did not improve from 0.33005
2674/2674 ————— 253s 95ms/step - accuracy: 0.6453 - loss: 0.2332 - val_accuracy: 0.5120 - val_loss: 0.3360 - learning_rate: 3.0000e-04
Epoch 4/15

2674/2674 ————— 0s 89ms/step - accuracy: 0.7164 - loss: 0.1850
Epoch 4: val_loss did not improve from 0.33005

Epoch 4: ReduceLROnPlateau reducing learning rate to 0.0001500000071246177.
2674/2674 ————— 253s 95ms/step - accuracy: 0.7164 - loss: 0.1850 - val_accuracy: 0.4850 - val_loss: 0.3741 - learning_rate: 3.0000e-04
Epoch 5/15

2674/2674 ————— 0s 89ms/step - accuracy: 0.7803 - loss: 0.1406
Epoch 5: val_loss did not improve from 0.33005
2674/2674 ————— 253s 95ms/step - accuracy: 0.7803 - loss: 0.1406 - val_accuracy: 0.4904 - val_loss: 0.4174 - learning_rate: 1.5000e-04
Epoch 6/15

2674/2674 ————— 0s 89ms/step - accuracy: 0.8087 - loss: 0.1218
Epoch 6: val_loss did not improve from 0.33005

Epoch 6: ReduceLROnPlateau reducing learning rate to 7.50000356230885e-05.
2674/2674 ————— 252s 94ms/step - accuracy: 0.8087 - loss: 0.1218 - val_accuracy: 0.4724 - val_loss: 0.4558 - learning_rate: 1.5000e-04
Epoch 7/15

2674/2674 ————— 0s 88ms/step - accuracy: 0.8277 - loss: 0.1073
Epoch 7: val_loss did not improve from 0.33005
2674/2674 ————— 251s 94ms/step - accuracy: 0.8277 - loss: 0.1073 - val_accuracy: 0.4679 - val_loss: 0.4899 - learning_rate: 7.5000e-05
Epoch 8/15

2674/2674 ————— 0s 88ms/step - accuracy: 0.8439 - loss: 0.0970
Epoch 8: val_loss did not improve from 0.33005

Epoch 8: ReduceLROnPlateau reducing learning rate to 3.750001781154424e-05.
2674/2674 ————— 250s 94ms/step - accuracy: 0.8439 - loss: 0.0970 - val_accuracy: 0.4665 - val_loss: 0.5186 - learning_rate: 7.5000e-05
Epoch 9/15

2674/2674 ————— 0s 87ms/step - accuracy: 0.8472 - loss: 0.0928
Epoch 9: val_loss did not improve from 0.33005
2674/2674 ————— 247s 92ms/step - accuracy: 0.8472 - loss: 0.0928 - val_accuracy: 0.4604 - val_loss: 0.5443 - learning_rate: 3.7500e-05
Epoch 10/15

2674/2674 ————— 0s 87ms/step - accuracy: 0.8545 - loss: 0.0884
Epoch 10: val_loss did not improve from 0.33005

Epoch 10: ReduceLROnPlateau reducing learning rate to 1.8750000890577212e-05.
2674/2674  **246s** 92ms/step - accuracy: 0.8545 - loss: 0.0884 - val_accuracy: 0.4584 - val_loss: 0.5563 - learning_rate: 3.7500e-05

Epoch 11/15
2674/2674  **0s** 87ms/step - accuracy: 0.8475 - loss: 0.0912
 Epoch 11: val_loss did not improve from 0.33005
2674/2674  **246s** 92ms/step - accuracy: 0.8475 - loss: 0.0912 - val_accuracy: 0.4555 - val_loss: 0.5491 - learning_rate: 1.8750e-05

Epoch 12/15
2674/2674  **0s** 87ms/step - accuracy: 0.8521 - loss: 0.0882
 Epoch 12: val_loss did not improve from 0.33005

Epoch 12: ReduceLROnPlateau reducing learning rate to 1e-05.
2674/2674  **246s** 92ms/step - accuracy: 0.8521 - loss: 0.0882 - val_accuracy: 0.4528 - val_loss: 0.5597 - learning_rate: 1.8750e-05

Epoch 13/15
2674/2674  **0s** 87ms/step - accuracy: 0.8435 - loss: 0.0927
 Epoch 13: val_loss did not improve from 0.33005
2674/2674  **246s** 92ms/step - accuracy: 0.8435 - loss: 0.0927 - val_accuracy: 0.4511 - val_loss: 0.5544 - learning_rate: 1.0000e-05

Epoch 14/15
2674/2674  **0s** 87ms/step - accuracy: 0.8455 - loss: 0.0911
 Epoch 14: val_loss did not improve from 0.33005
2674/2674  **246s** 92ms/step - accuracy: 0.8455 - loss: 0.0911 - val_accuracy: 0.4473 - val_loss: 0.5594 - learning_rate: 1.0000e-05

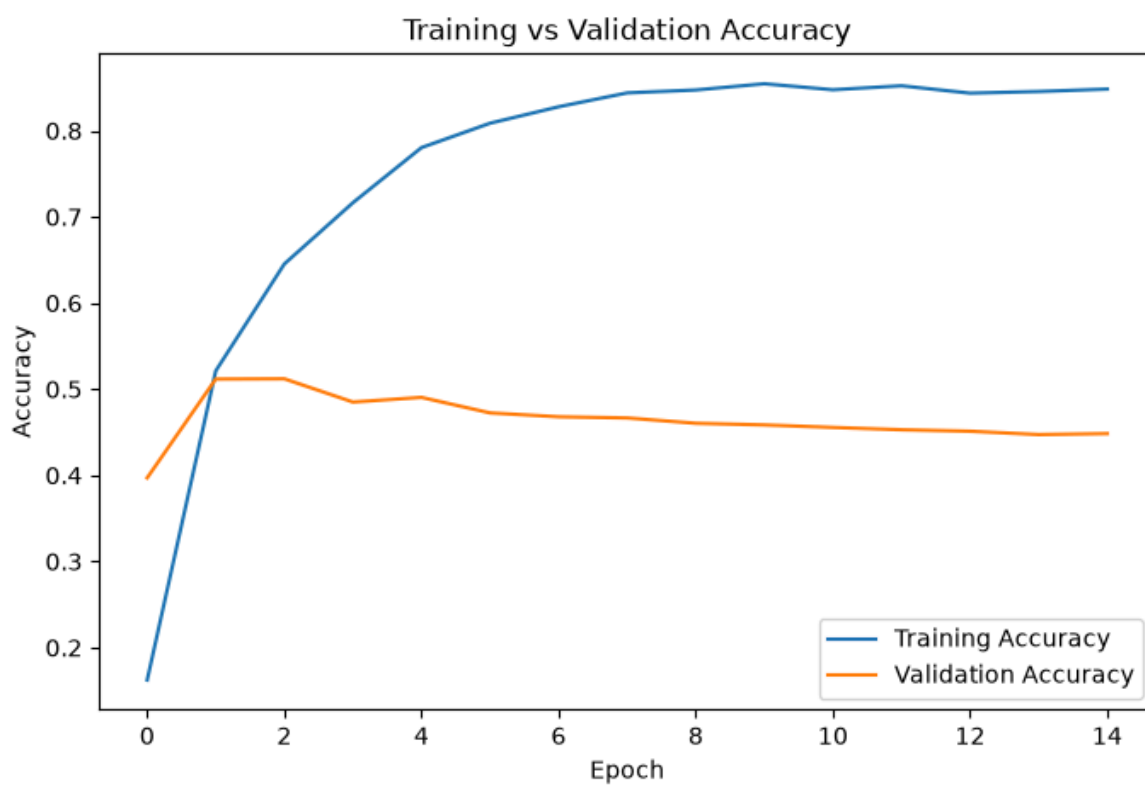
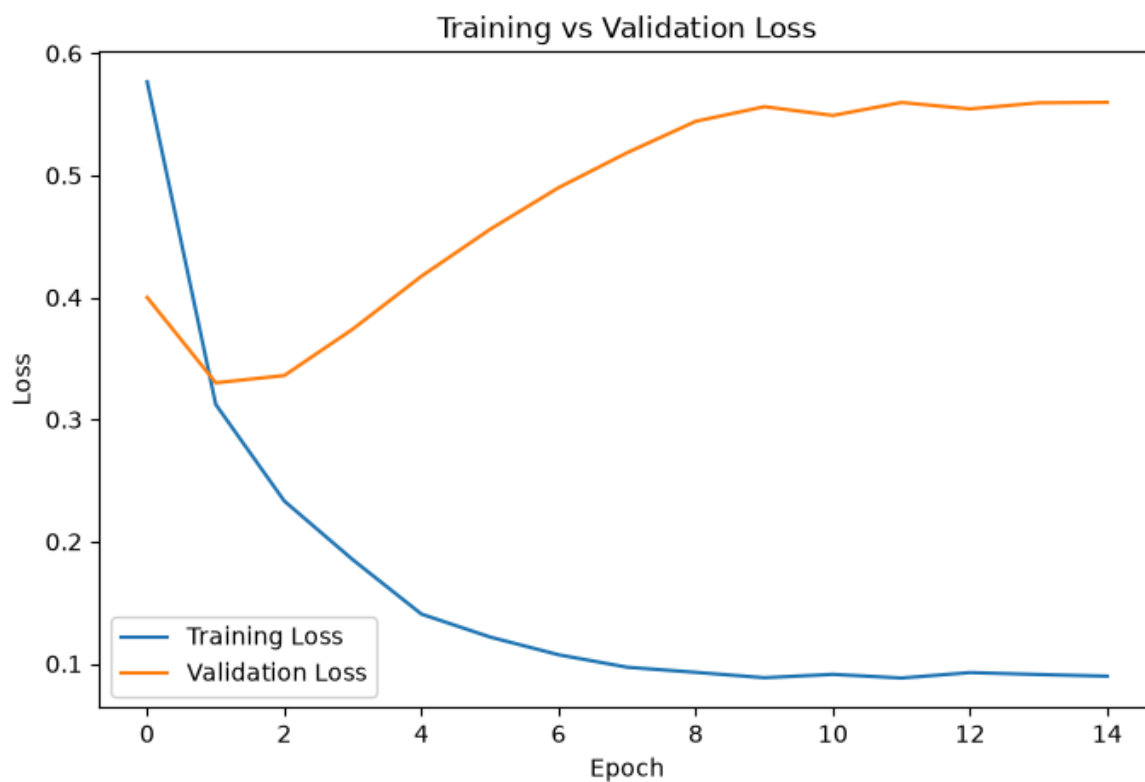
Epoch 15/15
2674/2674  **0s** 87ms/step - accuracy: 0.8483 - loss: 0.0897
 Epoch 15: val_loss did not improve from 0.33005
2674/2674  **246s** 92ms/step - accuracy: 0.8483 - loss: 0.0897 - val_accuracy: 0.4484 - val_loss: 0.5598 - learning_rate: 1.0000e-05

2.3 Training vs Validation Loss and Accuracy

```
In [ ]: plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()

best_epoch = int(np.argmin(history.history["val_loss"])) + 1
print("Epoch with minimum validation loss:", best_epoch)
print("Minimum validation loss:", min(history.history["val_loss"]))
```



Epoch with minimum validation loss: 2
Minimum validation loss: 0.33004942536354065

2.4 Greedy pointer answer generation

At inference time the decoder predicts **context positions**. The selected raw context tokens are copied directly, so names or rare words do not need to exist in the 50k output vocabulary. Generation stops when the special `END` pointer is selected.

```

In [ ]: def greedy_pointer_decode_batch(
        ctx_arr,
        q_arr,
        raw_context_tokens,
        batch_size=None
    ):
        """
        Decode one answer for each supplied context window.

        Returns
        -----
        predictions : list[str]
            Raw normalized context tokens copied from selected positions.
        confidences : list[float]
            Mean log-probability of selected pointer decisions.
        """
        n = len(ctx_arr)

        if batch_size is None:
            batch_size = n

        decoder_in = np.zeros(
            (
                n,
                MAX_ANSWER_LEN - 1
            ),
            dtype=np.int32
        )

        decoder_in[:, 0] = start_id

        generated = [
            []
            for _ in range(n)
        ]

        log_scores = [
            []
            for _ in range(n)
        ]

        finished = np.zeros(
            n,
            dtype=bool
        )

        true_context_lengths = np.array([
            min(
                len(tokens),
                MAX_CONTEXT_LEN
            )
            for tokens in raw_context_tokens
        ])

        for t in range(
            MAX_ANSWER_LEN - 1
        ):
            probs = qa_model.predict(

```

```

        [
            ctx_arr,
            q_arr,
            decoder_in
        ],
        batch_size=batch_size,
        verbose=0
    )

    step_probs = probs[:, t, :]

    next_ptr = np.argmax(
        step_probs,
        axis=-1
    )

    for i, ptr in enumerate(
        next_ptr
    ):
        if finished[i]:
            continue

        ptr = int(ptr)
        prob = float(
            step_probs[i, ptr]
        )

        log_scores[i].append(
            np.log(
                max(
                    prob,
                    1e-9
                )
            )
        )

    if ptr == END_POINTER_ID:
        finished[i] = True
        continue

    # Defensive protection; padded positions should already have
    # near-zero probability because pointer logits are masked.
    if ptr >= true_context_lengths[i]:
        finished[i] = True
        continue

    token = raw_context_tokens[i][ptr]
    generated[i].append(token)

    # Teacher-forcing replacement during inference:
    # feed the word found at the selected context position.
    if t + 1 < MAX_ANSWER_LEN - 1:
        decoder_in[
            i,
            t + 1
        ] = ctx_arr[
            i,
            ptr
        ]

```

```

        if np.all(finished):
            break

predictions = [
    " ".join(tokens).strip()
    for tokens in generated
]

confidences = [
    (
        float(np.mean(scores))
        if scores
        else -1e9
    )
    for scores in log_scores
]

return predictions, confidences

```

2.5 SQuAD-style Exact Match and token-level F1

```

In [ ]: def squad_normalize(s):
        def remove_articles(text):
            return re.sub(r"\b(a|an|the)\b", " ", text)

        def remove_punc(text):
            return "".join(ch for ch in text if ch not in set(string.punctuation))

        def white_space_fix(text):
            return " ".join(text.split())

        return white_space_fix(
            remove_articles(
                remove_punc(str(s).lower())
            )
        )

def exact_match_score(prediction, ground_truth):
    return float(squad_normalize(prediction) == squad_normalize(ground_truth))

def token_f1_score(prediction, ground_truth):
    pred_tokens = squad_normalize(prediction).split()
    gold_tokens = squad_normalize(ground_truth).split()

    common = Counter(pred_tokens) & Counter(gold_tokens)
    num_same = sum(common.values())

    if len(pred_tokens) == 0 or len(gold_tokens) == 0:
        return float(pred_tokens == gold_tokens)
    if num_same == 0:
        return 0.0

    precision = num_same / len(pred_tokens)
    recall = num_same / len(gold_tokens)
    return 2 * precision * recall / (precision + recall)

```

```

In [ ]: # -----
# FAIR FULL-CONTEXT EVALUATION WITH OVERLAPPING WINDOWS
# -----

EVAL_SAMPLES = min(
    500,
    len(val_df)
)

EVAL_STRIDE = 160
# MAX_CONTEXT_LEN=215 and stride=160 gives 55-token overlap,
# comfortably larger than the maximum retained answer length.

def make_sliding_windows(
    tokens,
    max_len=MAX_CONTEXT_LEN,
    stride=EVAL_STRIDE
):
    if len(tokens) <= max_len:
        return [
            (
                0,
                tokens
            )
        ]

    starts = list(
        range(
            0,
            len(tokens) - max_len + 1,
            stride
        )
    )

    last_start = len(tokens) - max_len

    if starts[-1] != last_start:
        starts.append(last_start)

    return [
        (
            start,
            tokens[
                start:start + max_len
            ]
        )
        for start in starts
    ]

eval_df = (
    val_df
    .iloc[:EVAL_SAMPLES]
    .copy()
    .reset_index(drop=True)
)

predictions = []

```

```

selected_scores = []
selected_window_starts = []
selected_context_ids = []
selected_context_tokens = []

eval_q = Xq_val[
    :EVAL_SAMPLES
]

for sample_idx in range(EVAL_SAMPLES):
    full_tokens = (
        eval_df.loc[
            sample_idx,
            "context_norm"
        ]
        .split()
    )

    windows = make_sliding_windows(
        full_tokens
    )

    window_starts = [
        start
        for start, _
        in windows
    ]

    raw_windows = [
        tokens
        for _, tokens
        in windows
    ]

    window_text = [
        " ".join(tokens)
        for tokens
        in raw_windows
    ]

    window_seq = ctx_tokenizer.texts_to_sequences(
        window_text
    )

    window_ids = pad_sequences(
        window_seq,
        maxlen=MAX_CONTEXT_LEN,
        padding="post",
        truncating="post"
    )

    q_single = eval_q[
        sample_idx:sample_idx + 1
    ]

    q_batch = np.repeat(
        q_single,
        len(window_ids),
        axis=0
    )

```

```

    )

    candidate_predictions, candidate_scores = (
        greedy_pointer_decode_batch(
            window_ids,
            q_batch,
            raw_windows,
            batch_size=max(
                1,
                min(
                    8,
                    len(window_ids)
                )
            )
        )
    )

    best_idx = int(
        np.argmax(
            candidate_scores
        )
    )

    predictions.append(
        candidate_predictions[
            best_idx
        ]
    )

    selected_scores.append(
        candidate_scores[
            best_idx
        ]
    )

    selected_window_starts.append(
        window_starts[
            best_idx
        ]
    )

    selected_context_ids.append(
        window_ids[
            best_idx
        ]
    )

    selected_context_tokens.append(
        raw_windows[
            best_idx
        ]
    )

    )

eval_ctx_selected = np.stack(
    selected_context_ids
)

eval_df["prediction"] = predictions
eval_df["window_start_token"] = selected_window_starts

```

```

eval_df["decode_log_confidence"] = selected_scores

eval_df["EM"] = [
    exact_match_score(
        p,
        g
    )
    for p, g in zip(
        eval_df["prediction"],
        eval_df["answer_norm"]
    )
]

eval_df["F1"] = [
    token_f1_score(
        p,
        g
    )
    for p, g in zip(
        eval_df["prediction"],
        eval_df["answer_norm"]
    )
]

overall_em = (
    100
    * eval_df["EM"].mean()
)

overall_f1 = (
    100
    * eval_df["F1"].mean()
)

print(
    f"Overall Exact Match (EM): {overall_em:.2f}%"
)

print(
    f"Overall Token-level F1: {overall_f1:.2f}%"
)

print(
    "\nEvaluation window selection used model confidence over overlapping "
    "full-context windows; gold answer positions were not used."
)

```

```

/home/cloud/nlp_gpu/lib64/python3.12/site-packages/keras/src/layers/layer.py:103
9: UserWarning: Layer 'cross_attention_logits' (of type Lambda) was passed an inp
ut with a mask attached to it. However, this layer does not support masking and w
ill therefore destroy the mask information. Downstream layers will not see the ma
sk.
    warnings.warn(
/home/cloud/nlp_gpu/lib64/python3.12/site-packages/keras/src/layers/layer.py:103
9: UserWarning: Layer 'cross_attended_context' (of type Lambda) was passed an inp
ut with a mask attached to it. However, this layer does not support masking and w
ill therefore destroy the mask information. Downstream layers will not see the ma
sk.
    warnings.warn(
/home/cloud/nlp_gpu/lib64/python3.12/site-packages/keras/src/layers/layer.py:103
9: UserWarning: Layer 'pointer_context_logits' (of type Lambda) was passed an inp
ut with a mask attached to it. However, this layer does not support masking and w
ill therefore destroy the mask information. Downstream layers will not see the ma
sk.
    warnings.warn(
/home/cloud/nlp_gpu/lib64/python3.12/site-packages/keras/src/layers/layer.py:103
9: UserWarning: Layer 'pointer_probabilities' (of type Lambda) was passed an inp
ut with a mask attached to it. However, this layer does not support masking and w
ill therefore destroy the mask information. Downstream layers will not see the mas
k.
    warnings.warn(
Overall Exact Match (EM): 3.40%
Overall Token-level F1: 8.39%

```

Evaluation window selection used model confidence over overlapping full-context w
indows; gold answer positions were not used.

2.6 Five target vs predicted answer samples

```

In [ ]: sample_cols = ["question_type", "question", "answer", "prediction", "EM", "F1"]
display(eval_df[sample_cols].head(5))

```

	question_type	question	answer	prediction	EM	F1
0	Who	Who was the first confederate soldier to be ki...	Private Henry Wyatt	battle of big bethel	0.0	0.0
1	What	What was Jan Joosten van Lodensteijn's Japanes...	Yayousu	dutch colleague of adams' station	0.0	0.0
2	What	What was the name of the Los Angeles-based tea...	Kiss	portland thunder	0.0	0.0
3	Who	Who wrote 'War and Peace'?	Leo Tolstoy	peter the great	0.0	0.0
4	When	When was the first Papal prounouncement agains...	28 April 1738	1917 code of canon law	0.0	0.0

```

In [ ]: # Automatic Task-2 observation / inference
train_loss_last = history.history["loss"][-1]
val_loss_last = history.history["val_loss"][-1]

print("OBSERVATION & INFERENCE - TASK 2")
print(f"- Final training loss: {train_loss_last:.4f}")

```

```

print(f"- Final validation loss: {val_loss_last:.4f}")
print(f"- Validation EM on {EVAL_SAMPLES} full-context samples: {overall_em:.2f}")
print(f"- Validation token F1 on {EVAL_SAMPLES} full-context samples: {overall_f1:.2f}")

if val_loss_last > train_loss_last * 1.20:
    print("- Validation loss is noticeably above training loss, suggesting some overfitting.")
else:
    print("- Training and validation losses remain reasonably close.")

if overall_f1 >= overall_em:
    print("- F1 is at least as high as EM because partial token overlap receives credit,")
    print("   whereas EM requires the complete normalized answer to match exactly.")

print("- The dual BiGRU encoders and 215x16 cross-attention create question-aware context states.")
print("- The autoregressive decoder then predicts context positions rather than a 50k vocabulary.")
print("- This copy/pointer formulation removes the closed-vocabulary answer bottleneck and greatly")
print("   reduces the number of output classes while retaining Sparse Categorical Cross-Entropy.")
print("- Final EM/F1 uses sliding windows over the full validation context without gold span locations.")

```

OBSERVATION & INFERENCE – TASK 2

- Final training loss: 0.0897
- Final validation loss: 0.5598
- Validation EM on 500 full-context samples: 3.40%
- Validation token F1 on 500 full-context samples: 8.39%
- Validation loss is noticeably above training loss, suggesting some overfitting.
- F1 is at least as high as EM because partial token overlap receives credit, whereas EM requires the complete normalized answer to match exactly.
- The dual BiGRU encoders and 215x16 cross-attention create question-aware context states.
- The autoregressive decoder then predicts context positions rather than a 50k vocabulary.
- This copy/pointer formulation removes the closed-vocabulary answer bottleneck and greatly reduces the number of output classes while retaining Sparse Categorical Cross-Entropy.
- Final EM/F1 uses sliding windows over the full validation context without gold span locations.

Task 3 — Question-Type Sensitivity & Alignment Interpretability

3.1 Performance breakdown for Who, Where, When, What

```

In [ ]: type_metrics = (
    eval_df.groupby("question_type")
    .agg(
        Samples=("question_type", "size"),
        EM=("EM", "mean"),
        F1=("F1", "mean")
    )
    .reset_index()
)

type_metrics["EM"] *= 100
type_metrics["F1"] *= 100

```

```
display(type_metrics.round(2))
```

	question_type	Samples	EM	F1
0	What	344	2.03	7.72
1	When	54	5.56	7.75
2	Where	32	0.00	3.66
3	Who	70	10.00	14.35

```
In [ ]: print("OBSERVATION & INFERENCE – TASK 3.1")

best_f1_row = type_metrics.loc[type_metrics["F1"].idxmax()]
best_em_row = type_metrics.loc[type_metrics["EM"].idxmax()]

print(
    f"- Highest F1 category: {best_f1_row['question_type']} "
    f"({best_f1_row['F1']:.2f}%)."
)
print(
    f"- Highest EM category: {best_em_row['question_type']} "
    f"({best_em_row['EM']:.2f}%)."
)

print("- 'When' questions often correspond to compact date/time expressions, whi
print(" 'Where' frequently target named entities. Such answers can be easier wh
print(" short, distinctive spans in the context.")
print("- 'What' questions are semantically broader and can require longer noun p
print(" descriptive spans, increasing the chance of partial rather than exact m
print("- The actual table above should be treated as the primary evidence; these
print(" explanations interpret the observed category differences rather than re
```

OBSERVATION & INFERENCE – TASK 3.1

- Highest F1 category: Who (14.35%).
- Highest EM category: Who (10.00%).
- 'When' questions often correspond to compact date/time expressions, while 'Who' and 'Where' frequently target named entities. Such answers can be easier when they form short, distinctive spans in the context.
- 'What' questions are semantically broader and can require longer noun phrases or descriptive spans, increasing the chance of partial rather than exact matches.
- The actual table above should be treated as the primary evidence; these linguistic explanations interpret the observed category differences rather than replacing them.

3.2 Attention map visualization for two validation samples

The cross-attention tensor has shape:

```
(batch, attention_heads, question_tokens, context_tokens)
```

For readability, the heatmap averages across heads and displays the most-attended context tokens.

```
In [ ]: def ids_to_tokens(ids, tokenizer):
    inv = {
        i: w
        for w, i
        in tokenizer.word_index.items()
    }

    return [
        inv.get(
            int(i),
            "<OOV>"
        )
        for i in ids
        if int(i) != 0
    ]

def plot_attention_for_sample(
    sample_idx,
    top_context_tokens=40
):
    # Use the same context chunk selected by full-context pointer decoding.
    ctx = eval_ctx_selected[
        sample_idx:sample_idx + 1
    ]

    q = eval_q[
        sample_idx:sample_idx + 1
    ]

    scores = attention_model.predict(
        [
            ctx,
            q
        ],
        verbose=0
    )[0]

    # (1, context_len, question_len)
    # -> (question_len, context_len)
    avg_scores = (
        scores
        .mean(axis=0)
        .T
    )

    q_ids = q[0]
    c_ids = ctx[0]

    q_nonzero = np.where(
        q_ids != 0
    )[0]

    c_nonzero = np.where(
        c_ids != 0
    )[0]
```

```

qlen = len(q_nonzero)
clen = len(c_nonzero)

mat = avg_scores[
    :qlen,
    :clen
]

q_tokens = ids_to_tokens(
    q_ids[:qlen],
    q_tokenizer
)

# Raw context labels preserve names even when the input id is <OOV>.
c_tokens_all = selected_context_tokens[
    sample_idx
][:clen]

importance = mat.max(
    axis=0
)

k = min(
    top_context_tokens,
    clen
)

top_idx = np.argsort(
    importance
)[-k:]

top_idx = np.sort(
    top_idx
)

mat_small = mat[
    :,
    top_idx
]

c_tokens = [
    c_tokens_all[i]
    for i in top_idx
]

plt.figure(
    figsize=(
        max(
            12,
            0.35 * len(c_tokens)
        ),
        max(
            4,
            0.55 * len(q_tokens)
        )
    )
)

plt.imshow(

```

```

        mat_small,
        aspect="auto"
    )

    plt.colorbar(
        label="Cross-attention weight"
    )

    plt.xticks(
        range(
            len(c_tokens)
        ),
        c_tokens,
        rotation=90
    )

    plt.yticks(
        range(
            len(q_tokens)
        ),
        q_tokens
    )

    plt.xlabel(
        "Most-attended context tokens"
    )

    plt.ylabel(
        "Question tokens"
    )

    plt.title(
        f"Cross-Attention Map - "
        f"{eval_df.loc[sample_idx, 'question_type']} question"
    )

    plt.tight_layout()
    plt.show()

    print(
        "Question :",
        eval_df.loc[
            sample_idx,
            "question"
        ]
    )

    print(
        "Target  :",
        eval_df.loc[
            sample_idx,
            "answer"
        ]
    )

    print(
        "Predicted:",
        eval_df.loc[
            sample_idx,
            "prediction"
        ]
    )

```

```

    ]
)

print(
    "Selected full-context window starts at token:",
    int(
        eval_df.loc[
            sample_idx,
            "window_start_token"
        ]
    )
)

print()

chosen = []

for qtype in [
    "Who",
    "Where",
    "When",
    "What"
]:
    idxs = eval_df.index[
        eval_df["question_type"]
        == qtype
    ].tolist()

    if idxs:
        chosen.append(
            idxs[0]
        )

    if len(chosen) == 2:
        break

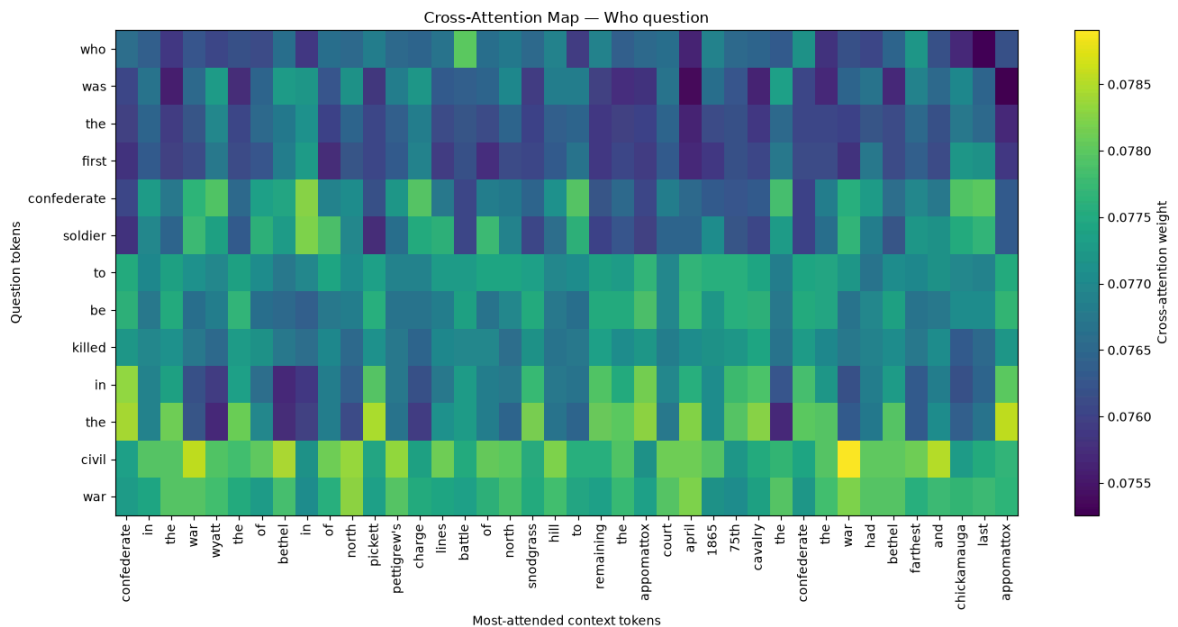
for idx in chosen:
    plot_attention_for_sample(
        idx
    )

```

```

/home/cloud/nlp_gpu/lib64/python3.12/site-packages/keras/src/layers/layer.py:103
9: UserWarning: Layer 'cross_attention_logits' (of type Lambda) was passed an input with a mask attached to it. However, this layer does not support masking and will therefore destroy the mask information. Downstream layers will not see the mask.
  warnings.warn(

```

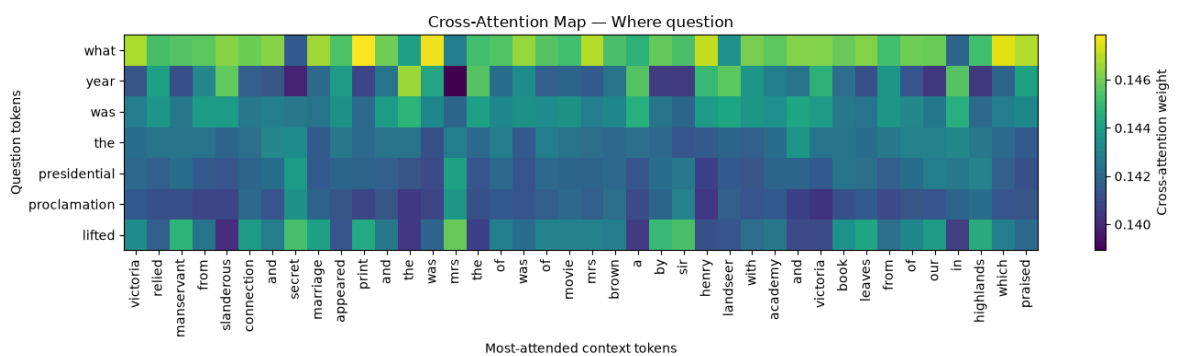


Question : Who was the first confederate soldier to be killed in the civil war?

Target : Private Henry Wyatt

Predicted: battle of big bethel

Selected full-context window starts at token: 0



Question : Where was John brown, Victorias manservant, from?

Target : Scotland

Predicted: john brown slanderous

Selected full-context window starts at token: 0

```
In [ ]: print("OBSERVATION & INFERENCE – TASK 3.2")
print("- Brighter/high-weight cells indicate stronger alignment between a question token and a context token.")
print("- Content-bearing question words and named/date/entity clues should place greater attention on context regions relevant to the predicted answer.")
print("- If attention is diffuse, the model is less certain about the location of supporting evidence; concentrated attention suggests a clearer token-level alignment.")
print("- Attention is an interpretability signal, not a proof of causal reasoning.")
```

OBSERVATION & INFERENCE – TASK 3.2

- Brighter/high-weight cells indicate stronger alignment between a question token and a context token.
- Content-bearing question words and named/date/entity clues should place greater attention on context regions relevant to the predicted answer.
- If attention is diffuse, the model is less certain about the location of supporting evidence; concentrated attention suggests a clearer token-level alignment.
- Attention is an interpretability signal, not a proof of causal reasoning.

Screenshot

The screenshot shows a Jupyter Notebook interface for an NLP assignment. The notebook is titled "NLP_assignment_2.ipynb" and is running on a Python 3.12 kernel. The left sidebar shows a file explorer with the following files:

Name	Modified
best_pointer_q...	2s ago
firefox.desktop	last yr.
jupyter-notebo...	last yr.
jupyterlab.desk...	last yr.
NLP_assignment...	now
train-v1.1.json	5m ago
Untitled.ipynb	6m ago
xfce-terminal-...	3y ago

The main code cell displays the following output:

```
Epoch 11: val_loss did not improve from 0.33005
2674/2674 246s 92ms/step - accuracy: 0.8475 - loss: 0.0912 - val_accuracy: 0.4555 - val_loss: 0.5491 - learning_rate: 1.8750e-05
Epoch 12/15
2674/2674 0s 87ms/step - accuracy: 0.8521 - loss: 0.0882
Epoch 12: val_loss did not improve from 0.33005

Epoch 12: ReduceLRonPlateau reducing learning rate to 1e-05.
2674/2674 246s 92ms/step - accuracy: 0.8521 - loss: 0.0882 - val_accuracy: 0.4528 - val_loss: 0.5597 - learning_rate: 1.8750e-05
Epoch 13/15
2674/2674 0s 87ms/step - accuracy: 0.8435 - loss: 0.0927
Epoch 13: val_loss did not improve from 0.33005
2674/2674 246s 92ms/step - accuracy: 0.8435 - loss: 0.0927 - val_accuracy: 0.4511 - val_loss: 0.5544 - learning_rate: 1.0000e-05
Epoch 14/15
2674/2674 0s 87ms/step - accuracy: 0.8455 - loss: 0.0911
Epoch 14: val_loss did not improve from 0.33005
2674/2674 246s 92ms/step - accuracy: 0.8455 - loss: 0.0911 - val_accuracy: 0.4473 - val_loss: 0.5594 - learning_rate: 1.0000e-05
Epoch 15/15
2674/2674 0s 87ms/step - accuracy: 0.8483 - loss: 0.0897
Epoch 15: val_loss did not improve from 0.33005
2674/2674 246s 92ms/step - accuracy: 0.8483 - loss: 0.0897 - val_accuracy: 0.4484 - val_loss: 0.5598 - learning_rate: 1.0000e-05
```

The status bar at the bottom indicates the notebook is in "Simple" mode, running on "Python 3.12 - NLP", with the cursor at "Ln 1, Col 1" in the file "NLP_assignment_2.ipynb".